R1.02 — Développement d'interfaces web : L'Architecture Client-Serveur et le Protocole HTTP

Thème principal : Fonctionnement du Web, Architecture Client-Serveur et Protocole HTTP

Synthèse exécutive : Ce cours pose les fondements indispensables du fonctionnement du Web en détaillant l'architecture client-serveur et le protocole HTTP (HyperText Transfer Protocol). L'étudiant y découvre comment un navigateur formule des requêtes standardisées pour obtenir des ressources auprès d'un serveur web, comment ce dernier y répond de manière asynchrone et sans état (stateless), et comment décoder les codes de statut HTTP pour diagnostiquer et déboguer efficacement des applications web.

L'Architecture Client-Serveur et le Principe Stateless

L'architecture client-serveur est le modèle de communication standard du Web. Contrairement aux réseaux pair-à-pair (P2P), les rôles sont strictement asymétriques : le client demande, le serveur exécute et répond. Le protocole HTTP est qualifié de 'stateless' (sans état), ce qui signifie que le serveur oublie instantanément le client une fois la réponse transmise. Bien que cela simplifie la scalabilité et la gestion des ressources, cela nécessite l'utilisation de mécanismes de contournement comme les cookies et les sessions pour maintenir une connexion utilisateur ou un panier d'achat.

- Client : Logiciel actif (navigateur comme Firefox ou Chrome) qui initie la communication en envoyant une requête.
- Serveur Web : Logiciel ou machine distant (Apache, Nginx, IIS) qui écoute passivement sur un port (ex: 80 ou 443) et traite les requêtes.
- Protocole Stateless (Sans état) : Caractéristique fondamentale où chaque échange requête-réponse est totalement indépendant, sans mémoire des interactions passées.

Cas concret enseignant : *Le professeur compare HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, il vous traite comme un parfait inconnu. Pour qu'il se souvienne de vous, vous devez lui présenter un badge (le cookie de session) à chaque phrase. Il ajoute également que lors d'une simple connexion au site de l'IUT de Bayonne, le navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes simultanées pour récupérer le HTML, les images, les feuilles de style CSS et les scripts JS.*

Structure des Requêtes, Réponses et Méthodes HTTP

Un message HTTP est purement textuel. Une requête se compose d'une ligne de commande, d'en-têtes, d'une ligne vide obligatoire, et d'un corps optionnel. La réponse du serveur adopte une structure similaire mais commence par une ligne de statut. Le choix de la méthode est crucial pour la sémantique de l'application web : GET expose les données dans l'URL et possède des limites de taille, alors que POST encapsule les données dans le corps de la requête de manière invisible dans l'URL.

- Ligne de requête / statut : Contient la méthode, l'URI et la version du protocole pour la requête, ou le code de statut et sa description pour la réponse.
- En-têtes (Headers) : Métadonnées sous forme de paires clé-valeur (Host, User-Agent, Content-Type) décrivant le contexte.
- Méthodes GET vs POST : GET est conçu pour la consultation de ressources (paramètres dans l'URL, sûre et idempotente), tandis que POST est conçu pour modifier l'état du serveur (paramètres dans le corps).

Cas concret enseignant : *Le professeur insiste sur le fait que le protocole HTTP est en clair (non chiffré par défaut). Utiliser la méthode GET pour transmettre un mot de passe est une faille de sécurité critique, car les paramètres apparaissent en clair dans l'historique du navigateur ou sur l'écran d'une personne se trouvant derrière l'utilisateur (shoulder surfing).*

La Classification des Codes de Statut HTTP

Les codes de statut à trois chiffres permettent au client d'analyser instantanément le résultat de sa requête. Il est fondamental pour un informaticien de différencier une erreur 4xx (problème de l'application cliente ou URL erronée à corriger) d'une erreur 5xx (bug dans le code serveur ou base de données inaccessible nécessitant une intervention sur l'infrastructure).

- Codes 2xx (Succès) : Indiquent que la requête a été traitée avec succès (ex: 200 OK).
- Codes 3xx (Redirection) : Indiquent qu'une action supplémentaire est requise par le client.
- Codes 4xx (Erreur Client) : Signalent une anomalie dans la requête du client (ex: 404 Not Found, 403 Forbidden).
- Codes 5xx (Erreur Serveur) : Signalent une défaillance interne du serveur malgré une requête valide (ex: 500 Internal Server Error).

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, il avait configuré par erreur une base de données qui plantait à chaque soumission de formulaire, renvoyant systématiquement une erreur 500. Les utilisateurs pensaient que le site était piraté alors que c'était simplement un point-virgule manquant dans son script PHP.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : Avec GET, les identifiants (nom d'utilisateur et mot de passe) s'afficheraient directement dans l'URL (ex: ?user=mat&pass;=123), ce qui est extrêmement dangereux pour la sécurité (historique, logs, épaules d'autrui). Avec POST, ces données voyagent cachées dans le corps (body) de la requête HTTP, à l'abri des regards dans la barre d'adresse.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante et identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer ce que le serveur va renvoyer si la ressource n'existe pas : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Analyser la première ligne : 'GET /images/logo.png HTTP/1.1'. Le mot 'GET' est la méthode HTTP de récupération de ressource. L'URI cible est '/images/logo.png'. 2. Identifier l'hôte cible en lisant l'en-tête 'Host: iutbayonne.univ-pau.fr', qui indique le domaine sollicité. 3. Analyser l'en-tête 'User-Agent: Mozilla/5.0' qui identifie le navigateur client. 4. Déduire le comportement en cas d'absence : si le fichier logo.png n'est pas présent sur le serveur, celui-ci renvoie un code de statut de la classe 4xx, précisément '404 Not Found'.

Résultat : Méthode : GET, Ressource : '/images/logo.png', Hôte cible : 'iutbayonne.univ-pau.fr', Code de retour en cas d'absence : 404 Not Found (Erreur Client).

■ Auto-Évaluation Flash d'Examen

Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP ?

R : C'est un protocole sans état où chaque échange requête-réponse est totalement indépendant, le serveur oubliant l'historique des interactions précédentes avec le client.

Q : Quelle est la différence majeure entre les codes d'erreur de la série 4xx et ceux de la série 5xx ?

R : Les codes 4xx indiquent une erreur provenant du client (URL erronée, syntaxe invalide), tandis que les codes 5xx signalent une défaillance interne du serveur (bug de code, base de données inaccessible).

Q : Pourquoi n'utilise-t-on jamais la méthode GET pour transmettre un mot de passe ?

R : Parce que la méthode GET transmet les paramètres directement dans l'URL, les rendant visibles dans l'historique du navigateur, les logs du serveur et aux yeux de tiers (shoulder surfing).